

Bancroft: HBM 밖 게놈 데이터를 위한 압축 경로

Bancroft: Genomics Acceleration Beyond On-Device Memory

Se-Min Lim, Seongyoung Kang, Sang-Woo Jun (2025)

압축한 데이터를 PCIe로 가져와 가속기 안에서 즉시 복원하는 genomics acceleration

왜 빠른 HBM이 있어도 큰 게놈 작업은 멈추는가

문제는 연산기 수만이 아니라, 작업 집합이 가속기 메모리에 들어가는가이다

- GPU·FPGA는 높은 계산 처리량과 빠른 on-board memory를 제공한다
- 하지만 실제 게놈 데이터 작업 집합은 보드 메모리 용량을 넘을 수 있다
- 넘친 데이터는 PCIe 또는 network를 통해 가져와야 하며 성능이 크게 떨어질 수 있다
- Bancroft의 목표는 큰 데이터를 압축 상태로 호스트에 두고 가속기에 공급하는 경로다

HBM의 ‘속도’가 높아도, HBM에 ‘들어가지 않는’ 데이터의 접근 경로는 별도로 설계해야 한다

용량과 대역폭은 다른 제약이다

Bancroft의 ‘beyond’는 HBM을 무한하게 만드는 말이 아니다

질문	HBM on device	host의 압축 DB	Bancroft의 역할
무엇이 들어가나	reference와 고정 buffer	압축 genomic pages	큰 원본을 HBM에 전부 두지 않음
어떤 제약인가	용량: 보드에 실을 수 있는 양	PCIe를 통한 이동	압축으로 이동할 원본량을 줄임
어떤 속도인가	resident reference의 높은 read BW	link BW와 page I/O	복원 후 user kernel에 넓은 stream 공급
말하면 안 되는 것	HBM이 큰 DB를 저장한다	PCIe가 사라진다	물리 link BW가 늘어난다

압축은 PCIe 물리 대역폭 대신 전송 byte당 원본 데이터 양을 키울 수 있다

Bancroft가 다루는 두 genomic stream

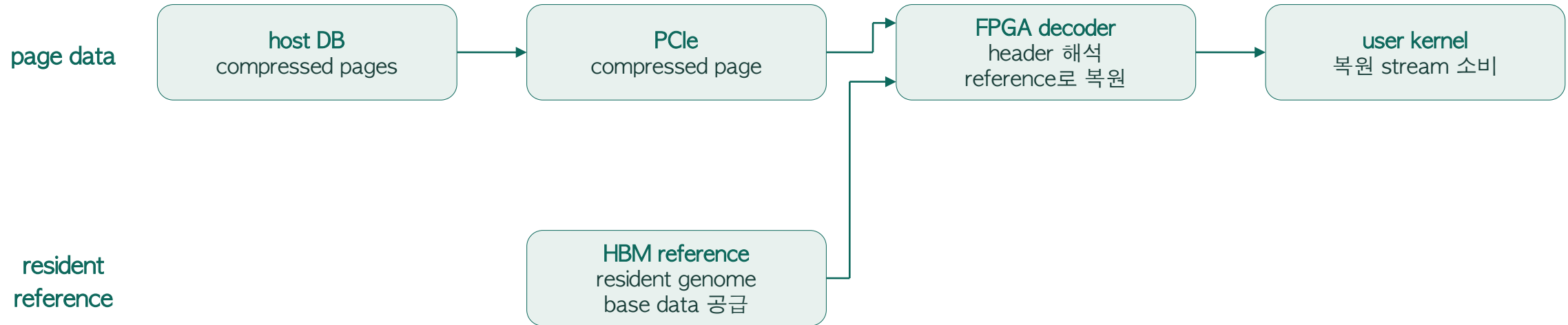
FASTA의 base와 FASTQ의 quality score는 같은 fixed-width 틀을 공유하지만 압축 근거는 다르다

stream	내용	Bancroft 방식	손실 여부
FASTA base read	a, c, g, t 염기열	압축 reference와 exact match	lossless
FASTQ quality score	각 base의 품질 점수	최근 값·p1/p2 mask	lossless
공통 하드웨어 단서	2-bit header + 32-bit payload	fixed-width grouped encoding	decoder 병렬화에 유리

base는 reference similarity, quality는 반복·빈도 특성을 이용한다

전체 경로: host의 압축 DB를 FPGA 안에서 복원한다

대용량 데이터는 host에, active reference와 decoder buffer는 HBM에 둔다



data는 host에서 오고, HBM은 decoder의 resident reference와 buffer를 맡는다

같은 ‘압축’도 FPGA와 host가 나눠서 한다

큰 random offset table은 FPGA HBM에 억지로 넣지 않는다

주체	말는 일	그 이유	결과
FPGA compressor	parse·stride shift·hash·filter	규칙적인 넓은 stream 처리	host로 후보 정보 전달
host software	cuckoo offset table 유지·조회·encode	human table은 최소 16 GB	큰 table을 host memory에 둬
FPGA decompressor	header parse·reference read·shuffle	fixed field와 HBM read	user kernel에 복원 stream
HBM PCs	reference와 고정 buffer	module마다 capacity를 차지	decompressor 수를 제한

Bancroft는 data structure 크기와 access pattern으로 FPGA·host 경계를 정한다

base read: 2-bit header가 다음 32-bit field를 정한다

header 16개를 묶고 payload는 고정 폭으로 두어 parser를 단순화한다

header	payload	decoder가 하는 일	다음 이동
00	있음	verbatim S bases	S
01	있음	forward match offset	K
10	있음	reverse-complement offset	K
11	없음	직전 match의 adjacent continuation	K

match length와 mismatch stride를 고정하면 variable-length parsing을 피할 수 있다

base compression: fixed K와 fixed S로 한 창씩 전진

복잡한 alignment가 아니라 exact K-mer candidate를 확인한다

- | | | |
|----|----------------|--------------------------------|
| 01 | K-mer 창 | target에서 길이 K 후보를 본다 |
| ↓ | | |
| 02 | cuckoo 후보 | forward·reverse offset 후보를 얻는다 |
| ↓ | | |
| 03 | reference 확인 | 후보가 exact match인지 읽어 비교한다 |
| ↓ | | |
| 04 | encode·advance | match는 K, 실패는 verbatim S |

hash lookup 자체가 정답 판정이 아니라, reference 확인 전의 후보 축소 단계다

quality score: reference 대신 최근 값과 p1/p2를 쓴다

6-bit score 다섯 개와 남은 2 bit를 이용한 fixed field encoding

header	payload	의미	조건
00	있음	verbatim S scores	새 값
01	없음	가장 최근 S-score list 반복	최근 list 필요
10	없음	가장 최근 score를 S회 반복	최근 score 필요
11	없음	p1/p2만으로 된 S-length string	mask로 p2 위치 지정

p1과 p2는 dataset sampling으로 정하는 상위 빈도 quality 값이며, 일반 상수가 아니다

설명용 trace: header가 어떤 종류의 복원을 고르는가

아래 K=4, S=2는 원문의 K=64, S=16이 아닌 이해용 축소 예다

입력 구간	선택 header	payload	복원 관점
reference와 4-base exact match	01	forward offset	reference에서 K=4를 읽음
다음 4-base가 인접 match	11	없음	직전 offset 다음 K=4
match 실패 2-base	00	verbatim 2 bases	payload를 그대로 붙임
quality가 p1/p2만 사용	11	p1/p2 mask	mask 위치만 p2로 바꿈

이 예는 header 의미를 보일 뿐, 압축률·cycle 수·실제 dataset 결과를 계산한 것이 아니다

decompression: page를 gap-free stream으로 만든다

fixed header grouping이 여러 decode와 reference request를 함께 진행하게 한다

- | | | |
|----|---------------|--------------------------------|
| 01 | page 수신 | compressed input을 buffer에 둔다 |
| ↓ | | |
| 02 | header parse | 4-wide sliding window로 종류를 읽는다 |
| ↓ | | |
| 03 | HBM reference | offset에 맞는 base를 fetch한다 |
| ↓ | | |
| 04 | shuffler | 빈틈 없는 512-bit output을 만든다 |

decoder의 HBM read가 random해도, fixed format과 buffer design이 output path를 구성한다

임의 위치 읽기: 원본 offset에서 compressed 4 KB page로

긴 파일에서도 host software가 file-position metadata를 관리한다

01 원본 file offset user가 읽고 싶은 위치



02 B+ tree offset을 compressed page로 매핑



03 4 KB page PCIe·NVMe 경로에서 읽는다



04 boundary 처리 긴 read 끝은 page padding

random access metadata는 decoder 내부가 아니라 host-side software interface에 있다

U50 prototype에서 module 수를 정하는 실제 자원

HBM pseudo-channel capacity가 여러 decompressor를 놓을 수 있는 수를 제한한다

항목	source 수치	설계에서의 뜻	주의
U50 HBM	8 GB; 32 PCs × 256 MB	module과 reference가 PC를 점유	평가 platform 수치
human reference	3 PCs	reference copy와 buffer 배치	species·reference에 따라 다름
decompressor × 1	3 PCs; 26 K LUT; 4 DSP	U50에서 최대 10개 언급	전체 chip 사용량 아님
compressor × 1	8 PCs; 36 K LUT; 480 DSP	hash/filter path	decompression과 역할 다름

‘10 decompressor’는 U50 32 PC와 human reference 조건의 upper bound다

결과를 읽기 전에: platform·dataset·baseline을 고정한다

서로 다른 축의 graph를 한 숫자로 합치지 않는다

축	논문 조건	무엇을 해석하나	혼동 금지
platform	U50, PCIe Gen3 × 16, 8 GB HBM	prototype traffic과 HBM 제약	다른 card의 결과로 일반화
datasets	human·mouse·maize, long-read 포함	compression characteristic 차이	모든 species가 동일함
ratio baseline	ASCII 또는 2-bit binary	압축 효율	GB/s throughput과 동치
comparison tools	quality 처리가 lossless가 아닐 수 있음	조건부 비교	동일 objective라고 가정

압축률·PCIe pressure·output throughput·chip usage는 서로 다른 단위를 가진 결과다

K와 S는 hardware-friendly 형식의 경험적 선택이다

고정 폭은 decode를 쉽게 하지만 압축률에는 dataset·parameter 영향이 남는다

parameter	역할	source observation	발표 해석
S	verbatim stride	S=16이 Fig. 6에서 선택됨	작으면 후보를 더 자주 확인
K	exact match length	human은 K=64; maize는 K=128이 유리할 수 있음	한 값이 모든 data에 최적 아님
quality S	score field length	source는 S=5 사용	5×6-bit payload와 연결
losslessness	base·quality 복원	Bancroft는 lossless로 제시	타 tool의 setting 확인 필요

K=64, S=16은 논문의 prototype defaults이며 수학적 최적값 선언이 아니다

측정 1 : 한 decompressor가 output bus에 가깝게 간다

Fig. 10은 dataset별 decompressed throughput과 PCIe pressure를 함께 보여 준다

- source는 한 decompressor가 테스트한 모든 dataset에서 512-bit output bus를 거의 포화한다고 설명한다
- 동시에 PCIe에 가해지는 pressure는 낮게 나타나며, 이것이 compression의 직접 목적이다
- 이 결과는 U50 prototype과 Fig. 10의 FASTQ dataset 조건이다
- ‘거의 포화’는 모든 system-reference access pattern의 보편 상한이 아니다

compressed transfer가 충분히 작을 때 PCIe는 decoder output의 즉시 병목이 아닐 수 있다

측정 2: 여러 stream을 병렬로 복원할 때

네 decompressor의 ensemble은 Fig. 11에서 네 channel output peak 근처로 간다

- ensemble 이름은 reference 수 R과 long-read dataset 수를 조합해 나타낸다
- 논문은 2R2와 1R3 등 현실적 ensemble workload를 구성한다
- Fig. 11의 총 throughput은 네 decompressor의 합이며, source는 peak bandwidth 근처라고 해석한다
- dataset species는 이 실험에서 compression characteristic을 주는 real data로 사용된다

병렬 channel 수를 늘려도 HBM PC-reference capacity-workload mix가 함께 설계 조건이 된다

측정 3: U50 HBM 전력 제한에 닿는다

Fig. 12의 약 70 GB/s는 PCIe가 아니라 U50의 HBM power throttling 한계로 설명된다

항목	source 설명	의미	말하면 안 되는 것
workload	HG002를 여러 reference에 align	favorable multi-reference case	모든 workload의 peak
관측	약 70 GB/s에서 stall	U50 measured result	HBM nominal peak
원인	HBM power throttled at 7.8 W	보드 특유의 제약	Bancroft algorithm 일반 한계
source framing	power-throttled U50의 sustained 성능 약 34%	DDR4 U250 peak와 유사	unthrottled 결과

PCIe 병목이 줄면 HBM의 실제 power-thermal operating point가 다음 병목이 될 수 있다

U55C 결과는 ‘측정’이 아니라 조건부 projection이다

전력 throttling이 없는 device와 10 decompressor를 가정한 Fig. 13

가정	paper projection	왜 가능한가	판독 규칙
device	non-throttled Alveo U55C	available HBM BW 약 512 GB/s	U50 measurement로 표기하지 않기
parallelism	up to 10 decompressor	reference HBM capacity가 한계	단일 module 재측정 아님
output	total 160 GB/s	10 module ensemble	Fig. 13 projection
comparison	peak HBM 31%; DDR4의 거의 2배	source의 stated baseline	다른 card·app으로 일반화 금지

Fig. 13은 architecture scaling argument이지 U55C에서 이미 얻은 benchmark가 아니다

compression: FPGA offload 뒤에도 host lookup은 남는다

Fig. 14는 partition을 바꿔도 random cuckoo access가 host bottleneck이 될 수 있음을 보인다

01 software only thread 증가에 따라 compute가 병목



02 hash-encoder offload FPGA가 parse와 hash를 처리



03 host random lookup 32-bit cuckoo access의 cache miss



04 probabilistic filter 불필요한 host pressure를 줄인다

data path 일부를 offload해도 남은 random access가 전체 throughput을 지배할 수 있다

accelerator 비교는 숫자보다 조건을 먼저 읽는다

Fig. 15는 최고 속도 하나가 아니라 platform·memory·normalization의 차이를 포함한다

비교 축	Bancroft 설정	논문이 밝힌 caveat	안전한 결론
FPGA compression	single channel; B73 worst case	FPGA platforms가 다름	raw LUT만으로 공정성 판단 불가
host memory	cuckoo hash를 host memory에 둠	기존 설계와 resource boundary가 다름	chip area만 비교하지 않기
GPU	single GPU로 normalization	multi-GPU result도 포함	hardware budget이 같지 않을 수 있음
reported trend	Arram15 약 100× ; Chen23 30× 초과	source 비교 조건 하의 수치	일반 speedup 보장 아님

raw LUT comparison은 부정확하므로, 수치가 같은 resource budget의 증명은 아니다

case study: SHD filtering에 stream을 공급한다

Bancroft 자체가 alignment algorithm은 아니며, memory-intensive filtering의 data path를 돕는다

01 candidate read seed 후 많은 후보를 받는다



02 shifted references 여러 shift의 Hamming mask 생성



03 mask combine AND 뒤 one count로 filter



04 Bancroft stream compressed large data를 decoder가 공급

이 사례의 이점은 filter algorithm과 대규모 data delivery를 합친 end-to-end 문맥에 있다

Fig. 17의 두 Bancroft 점은 같은 종류의 결과가 아니다

측정된 U50와 non-throttled ideal projection을 분리해야 한다

label	상태	논문 설명	발표에서의 표기
B-R	real-world U50	HBM power throttled 상태	측정
B-I	ideal U50	HBM power throttling 제거	projection
GPU baselines	GTX 1080 Ti / RTX 3080 Ti	각 paper의 platform	동일 device 시험 아님
figure annotation	$7.5 \times$, $21 \times$	해당 comparison placement	일반적 보장으로 말하지 않기

application comparison에서 measured와 ideal point를 합치면 evidence level이 바뀐다

어디까지가 Bancroft의 주장인가

architecture insight는 강하지만, 모든 large-data accelerator에 대한 자동 해법은 아니다

경계	source가 보여 준 것	이 자료에서 보류하는 주장
platform	Alveo U50 prototype과 특정 HBM/PCIe 조건	mobile-edge device 일반 deployment
capacity	compressed host data를 on-demand로 공급	HBM 또는 storage가 무한함
compression	reference similarity와 data 특성에 의존	모든 genome의 같은 ratio-throughput
comparison	조건이 적힌 graph와 case study	동일 자원·동일 accuracy의 보편 우위
application	SHD filtering case	모든 pipeline의 end-to-end 보장

Bancroft의 핵심은 큰 data를 쓰게 하는 압축 access path이며, 수치는 조건부 실증이다

정리: 압축 format이 memory hierarchy의 일부가 된다

Bancroft는 compression ratio, fixed-width decoder, HBM reference, host software를 함께 맞춘다

- 큰 genomic DB는 host에 compressed form으로 유지한다
- fixed K/S와 grouped header가 FPGA decoder의 규칙적인 data path를 만든다
- HBM은 전체 DB가 아니라 resident reference와 module buffer에 쓴다
- 측정·projection·normalized comparison을 구분해야 architecture claim을 정확히 읽을 수 있다

beyond on-device memory는 압축으로 host data와 accelerator compute를 연결하는 원리다